# Football Intelligence Platform

What we built, in plain language - with the file names

**What the project does.** It answers two questions about football, using the record of every single touch in a season:

- **Who plays like this player?** Pick a player, get a ranked list of others who do similar things - plus the reasons why.
- **Which teams play alike?** Group the 20 teams by their style of play, and show what makes each group different.

It deliberately does not predict match results. It describes what happened, which is what a scout or analyst actually needs.

### The data

StatsBomb Open Data for the Premier League 2015/16 season: 380 matches and 1,313,773 individual events (every pass, shot, tackle and pressure, each with pitch coordinates). It is free and legal to use with attribution. We chose depth over recency: newer free data has only summary totals, which cannot support the spatial features this project is built on.

Repository: github.com/SamiRadulov/football-intelligence-platform

## 1. The big picture: data flows downhill

Everything is built as a chain of layers. Each layer only reads from the one above it and never edits it. If a number looks wrong, you can walk back up the chain to find where it came from.

| Layer | What lives there | Where |
|---|---|---|
| 1. Raw | The original StatsBomb JSON files, completely unchanged | `data/raw/` |
| 2. Staging | The same data flattened into clean tables (Parquet files) | `data/staging/` |
| 3. Curated | Those tables loaded into a database that can be queried | `data/curated.duckdb` |
| 4. Marts | The finished per-player and per-team measurements | `tables inside that database` |
| 5. Models | Similarity and the team style groups | `src/similarity/`, `src/clustering/` |
| 6. App | The website that shows all of it | `app.py, pages/` |

The build stops if any of 15 hard quality checks fails, so the finished measurements can never be built from data that breaks them.

## 2. What was built, phase by phase

### Phase 0 - Choosing the data

Every free competition-season was audited before anything was built. The audit is saved so the choice can be re-examined.

| File | What it does |
|---|---|
| `scripts/data_audit.py` | Counts what each dataset actually contains |
| `docs/data_audit.csv` | The audit result |

| File | What it does |
|---|---|
| `artifacts/feature_config.yml` | Every analytical setting in one editable file, so choices are reviewable without reading code |

### Phase 1 - Getting the data

| File | What it does |
|---|---|
| `src/ingestion/statsbomb_open_data.py` | Downloads one file, safely and repeatably |
| `scripts/download_data.py` | Downloads the whole season |
| `data/raw/manifest.json` | A record of exactly what was downloaded and when |

Downloads are idempotent: a file is written to a temporary name and renamed only when complete, so an interrupted run never leaves a half-file behind.

### Phase 2 - Cleaning it up

| File | What it does |
|---|---|
| `src/transformations/matches.py` | The fixture list |
| `src/transformations/players.py` | The player list |
| `src/transformations/minutes.py` | Who was on the pitch, and for how long |
| `src/transformations/events.py` | 1.3 million events into one flat table |
| `src/transformations/names.py` | Repairs a spelling fault in the source data |
| `src/validation/checks.py` | The 15 quality checks |
| `scripts/build_staging.py` | Runs all of the above |

### Phase 3 - Measuring players

| File | What it does |
|---|---|
| `src/features/roles.py` | Sorts players into five position groups |
| `src/features/player_match.py` | What each player did in each match |
| `src/features/player_season.py` | Season totals turned into fair per-90 rates |
| `scripts/build_player_features.py` | Runs it |

Everything is compared **within a position group**. Forty passes per 90 minutes means one thing for a centre-back and something else for a winger, so a centre-back is only ever ranked against other centre-backs.

### Phase 4 - Finding similar players

| File | What it does |
|---|---|
| `src/similarity/feature_matrix.py` | Puts every player on a comparable scale |
| `src/similarity/engine.py` | The similarity calculation and its explanation |
| `scripts/find_similar.py` | Ask a question from the command line |
| `scripts/validate_similarity.py` | Proves the answers are stable |

Two rules make the answers defensible. Candidates are filtered by position and minutes **before** anything is scored, so an unsuitable player cannot appear by statistical fluke. And every result comes with the reasons: where the two players match, and where they differ.

> It works: Kante returns Gueye, Cork and Dembele. Ozil returns De Bruyne, Silva and Payet.

### Phase 5 and 6 - Measuring and grouping teams

| File | What it does |
|---|---|
| `src/features/pitch.py` | Shared pitch geometry, so player and team definitions cannot drift apart |
| `src/features/team_match.py` | 28 style measurements per team per match |
| `src/features/team_season.py` | Season averages, and how much each varies |
| `src/clustering/style_model.py` | The grouping maths |
| `scripts/build_team_clusters.py` | Run with --explore first to inspect the options, then again to save the model |

The groups were named only after looking at what defined them, never decided in advance.

| Style | Teams | What defines them |
|---|---|---|
| Possession-dominant high press | Arsenal, Chelsea, Liverpool, Man City, Man Utd, Spurs | Keep the ball, play high up the pitch, press aggressively, few long balls |
| Direct and wide | Palace, Leicester, Norwich, Southampton, Sunderland, Watford, West Brom, West Ham | Move the ball forward quickly, many long passes and crosses, less possession |
| Deep build-up, low territory | Bournemouth, Villa, Everton, Newcastle, Stoke, Swansea | Play in their own half, pass backwards more, rarely reach the final third |

### The honest caveat we wrote down

The first group is exactly the big six. That means style and quality are tangled together, because better teams naturally have more possession. We state this openly rather than waiting for someone to spot it. The counter-evidence: the direct and wide group contains both the champions (Leicester) and two relegated sides, so the groups are not simply the league table.

### Phase 7 and 8 - The app, and putting it online

Eight pages, reachable from one search box.

| File | The page |
|---|---|
| `app.py` | The entry point - start the app with this |

| File | The page |
|---|---|
| `pages/home.py` | One search box over players, clubs and fixtures |
| `pages/player_search.py` | Find similar players |
| `pages/player_profile.py` | One player: twelve percentile rings, or a report-style table |
| `pages/player_comparison.py` | Compare two players side by side |
| `pages/team_profile.py` | One club: its record, its style, its league-leading players |
| `pages/team_style_map.py` | The map of team styles |
| `pages/matches.py` | All 380 fixtures, and a full report for any one of them |
| `pages/methodology.py` | How it works, and the limitations |
| `src/app/search.py` | Search that accepts names as they are typed |
| `src/app/data.py` | Loads data once and remembers it, so the app stays fast |
| `src/app/charts.py` | The charts |

Charts show percentiles rather than statistical scores, because "84th percentile among centre-backs" means something to a human and "z = +1.02" does not.

> Search works in plain English. Apostrophes and accents are optional, because they are stripped from both the query and the name before matching: "Ngolo Kante" is an exact match for the stored "N'Golo Kante", and "spurs" finds Tottenham Hotspur.

| File | What it does |
|---|---|
| `scripts/build_app_db.py` | Makes a small 3.4 MB database holding only the finished results |
| `data/sample/app.duckdb` | That database - the only data file stored in the repository |
| `.streamlit/config.toml` | The colours and text sizes |
| `.github/workflows/ci.yml` | Runs the tests automatically on every change |
| `scripts/capture_screenshots.py` | Regenerates the README screenshots |

> Because the raw data is not stored in the repository, the automatic test run has to use that small database - which means every test run is also a test of exactly what gets deployed.

## 3. How we know it works

There are 168 automatic tests. They run in about a minute and check the things that are easy to get quietly wrong.

| Test file | What it protects |
|---|---|
| `tests/test_ingestion.py` | The download addresses are correct |
| `tests/test_minutes.py` | Minutes are counted correctly, including the injury-gap bug |
| `tests/test_events.py` | Events are flattened correctly, including the aerials bug |
| `tests/test_names.py` | The spelling repair fixes the fault and nothing else |
| `tests/test_roles.py` | Players land in the right position group |
| `tests/test_player_match.py` | The football definitions: progressive pass, box entry, penalties excluded |
| `tests/test_similarity.py` | The similarity maths behaves sensibly |
| `tests/test_team_match.py` | Team measurements, including the set-piece fix |

| Test file | What it protects |
|---|---|
| `tests/test_clustering.py` | Grouping finds groups that really are there |
| `tests/test_search.py` | Names typed without accents still find the player |
| `tests/test_insights.py` | The league table, and the top-three rule |
| `tests/test_app_smoke.py` | Every page of the app loads without crashing |

### The pattern worth noticing

Four real bugs were found in this project: minutes counting injury gaps, aerial duels missing entirely, set pieces defined too loosely, and ten player names arriving with doubled apostrophes. None of them were found by staring at code - the code ran fine and produced the wrong number. Each was caught by asking **does this look like football?** and then investigating instead of assuming. That habit is the single most valuable thing here.

## 4. The commands

Run these from the project folder. Steps 1 to 5 only need running once.

```
cd "C:\Users\user\Documents\Claude\Projects\Football Intelligence platform"
```

| Step | Command |
|---|---|
| 1. Download the data (~1.2 GB, one-off) | `.venv\Scripts\python scripts\download_data.py` |
| 2. Build the clean tables | `.venv\Scripts\python scripts\build_staging.py` |
| 3. Build the player metrics | `.venv\Scripts\python scripts\build_player_features.py` |
| 4. Build the team style metrics | `.venv\Scripts\python scripts\build_team_features.py` |
| 5. Group the teams by style | `.venv\Scripts\python scripts\build_team_clusters.py` |
| Start the app | `.venv\Scripts\streamlit run app.py` |
| Ask a similarity question | `.venv\Scripts\python scripts\find_similar.py "Kante"` |
| Run all the tests | `.venv\Scripts\python -m pytest -q` |

## 5. The written documents

| File | What it contains |
|---|---|
| `README.md` | The front page: what it is and how to run it |
| `METHODOLOGY.md` | How the models work, the validation results, and the limitations |
| `docs/MODEL_CARD.md` | The player similarity model: what to use it for, and where it fails |
| `docs/CLUSTER_CARD.md` | The team style groups: how they were chosen, and their limits |
| `docs/METRIC_DICTIONARY.md` | Every metric defined precisely, with its denominator |
| `docs/DECISIONS.md` | Why the architecture is what it is - the alternatives weighed, and what each choice cost |
| `AI_USAGE.md` | How this was built with an AI assistant, including what it got wrong |
| `DATA_SOURCES.md` | Where the data comes from and the attribution required |
| `artifacts/feature_config.yml` | All the analytical settings in one editable file |

### The one thing to remember

The value of this project is not that it uses clever maths. It is that every number can be traced back to a definition, every definition is written down, and every claim has been checked against what actually happened on the pitch. That is what makes it defensible when someone asks you about it.